AMMAN ARAB UNIVERSITY
جامعة عمان العربية

College of Information Technology
كلية تكنولوجيا المعلومات
قسم هندسة البرمجيات

Flutter and Dart Programming Project

Noor Al-Iman: Quran Memorization Center App

Phase 1 Report: Plan & Design

Students

Adam Alrefa'e	**202320335**
Laith Al Haddad	**202320142**
Yousef Hassan	**202310045**

Supervisor: **Ala Ali Hamel Abu Thawabeh**

A project report submitted in partial fulfillment of the requirements
for Special Topics in Software Engineering
Amman - Jordan
2026/2027

Abstract

Noor Al-Iman is a Quran memorization center where students attend classrooms, memorize Surahs with a teacher, have their attendance taken every day and receive gifts as a reward for their effort. Most of this work is still recorded on paper, which makes it slow to follow a student's progress, easy to lose attendance sheets and hard for supervisors to know how each classroom is doing. This project proposes a mobile application, built with Flutter, that moves the daily work of the center into one shared system. The app serves three roles: admins, who manage students, classrooms and the gift catalog; teachers, who take attendance, record memorization progress and give gifts; and supervisors, who monitor classrooms through reports. All data is stored in a MySQL database and reached through a secure PHP REST API, so every staff member sees the same up-to-date information. The project follows the Scrum framework across three stages. This report covers the first stage: the project proposal and requirements, the UI/UX design, and the architecture and project set-up.

Table of Contents

Abstract II
Chapter 1: Introduction 1
1.1 Overview 1
1.2 System Description 1
1.3 Objectives 1
1.4 Scope 2
1.5 Project Proposal 2
Chapter 2: Methodology 4
2.1 Overview 4
2.2 Choosing a Development Approach 4
2.3 Team Organization 5
2.4 Scrum Events 5
2.5 Sprint Plan and Schedule 5
2.6 Tools 6
2.7 Definition of Done 7
Chapter 3: Requirements Analysis 8
3.1 Overview 8
3.2 Requirements Gathering 8
3.3 Stakeholders and Users 8
3.4 Use Case Model 8
3.5 User Stories 9
3.6 Functional Requirements 11
3.7 Non-Functional Requirements 12
3.8 Feature List and Scope 13
3.9 Assumptions and Constraints 13
3.10 Task Backlog 13
Chapter 4: UI/UX Design 15
4.1 Overview 15
4.2 Design Goals 15
4.3 Screen Inventory 15
4.4 Navigation Flow 16
4.5 Screen Designs 17
4.6 Colours 23
4.7 Typography 24
4.8 Components and Usability Decisions 25
Chapter 5: Architecture and Project Set-up 27

5.1 Overview 27
5.2 System Architecture 27
5.3 Technology Stack 28
5.4 State Management 28
5.5 Data Model 29
5.6 API Design 31
5.7 Folder Structure 32
5.8 GitHub Repository and Workflow 32
Chapter 6: Conclusion 33
6.1 Summary 33
6.2 Next Steps 33
References 34
Appendix A: Diagram Source Files 34

List of Figures

Figure 1: Scrum process followed by the team 5
Figure 2: Gantt chart of the project schedule 6
Figure 3: Use case diagram 9
Figure 4: Navigation flow between screens 16
Figure 5: S001 Splash 17
Figure 6: S002 Login 17
Figure 7: S003 Admin Dashboard 17
Figure 8: S004 Teacher Home (My Classroom) 18
Figure 9: S005 Supervisor Dashboard 18
Figure 10: S006 Students List 18
Figure 11: S007 Add / Edit Student 19
Figure 12: S008 Student Profile 19
Figure 13: S009 Classrooms List 19
Figure 14: S010 Classroom Details 20
Figure 15: S011 New / Edit Classroom 20
Figure 16: S012 Take Attendance 20
Figure 17: S013 Add Progress 21
Figure 18: S014 Give Gift 21
Figure 19: S015 Gift Catalog 21
Figure 20: S016 Reports 22
Figure 21: S017 Profile & Settings 22
Figure 22: S018 Delete Check Dialog 22
Figure 23: Examples of error and empty states 23
Figure 24: Teacher Home in light, dark, English and Arabic 23
Figure 25: Colour palette (light theme) 24
Figure 26: Type scale in English and Arabic 25
Figure 27: Shared components 26
Figure 28: System architecture and data connection 27
Figure 29: EER diagram of the Noor Al-Iman database 30

List of Tables

Table 1: Project proposal summary 3
Table 2: Comparison of development approaches 4
Table 3: Scrum roles in the team 5
Table 4: Scrum events adapted to the team 5
Table 5: Sprint plan 6
Table 6: Stakeholders of the system 8
Table 7: User stories 10
Table 8: Functional requirements 11
Table 9: Non-functional requirements 12
Table 10: Feature list 13
Table 11: Task backlog 14
Table 12: Screen inventory 15
Table 13: Bottom navigation items by role 16
Table 14: Colour usage 24

Table 15: Type scale 25
Table 16: Technology stack 28
Table 17: Comparison of state management approaches 29
Table 18: Database tables 30
Table 19: Integrity rules enforced by the database 31
Table 20: Main API endpoints 31

Chapter 1: Introduction

1.1 Overview

Quran memorization centers play an important role in the community. Students of different ages come several times a week, sit with a teacher in a small classroom and work through the Quran Surah by Surah. Behind this simple routine there is a lot of record keeping: who attended today, which verses each student recited, how well they recited them, which students deserve a reward and whether a classroom has room for a new student.

Noor Al-Iman is one of these centers. Its staff are teachers and supervisors who care about their students, but the tools they use have not changed for years: notebooks, printed attendance sheets and memory. This project builds a mobile application that supports the staff in this daily work without changing the way they teach.

1.2 System Description

1.2.1 Problem Statement

At Noor Al-Iman, each teacher keeps a personal notebook with the progress of every student. When a teacher is absent, the substitute has no reliable way to know where a student stopped, so students sometimes repeat Surahs they already finished or skip parts by mistake. Attendance is taken on paper sheets that are often lost or filled in late, and at the end of the month somebody has to count the sheets by hand to find students with too many absences.

Gifts are given to motivate students, but nobody records who received what. Some students end up receiving several gifts while others are forgotten. Supervisors, who are responsible for several classrooms, can only find out how a classroom is doing by visiting it and asking the teacher. Finally, new students are sometimes added to classrooms that are already full, because the capacity of each room is not checked at the time of enrollment.

The center needs a single system, available on the phones staff already carry, that keeps all of this information in one place and applies the center's rules automatically.

1.2.2 Proposed Solution

We propose a Flutter mobile application for the center's staff. The app connects to a MySQL database through a PHP REST API, so that a change made by one user, such as a new student added by the admin, is visible to every teacher and supervisor the next time their screen loads. Each user signs in with a role, and the app shows only the functions of that role.

1.3 Objectives

The main objective of this project is to replace the paper-based records of the Noor Al-Iman center with a reliable mobile system that is fast to use during class and gives supervisors a clear view of every classroom. The specific objectives are:

1. Provide a central, shared database of students, classrooms, curriculum, attendance, memorization progress and gifts, reachable from any staff phone.
2. Allow a teacher to take attendance for a full classroom in under one minute, and to record a memorization session (Surah, verse range and evaluation) in a few taps.
3. Give every student a single profile that shows their progress, attendance history and gifts, so any teacher can continue where another stopped.
4. Enforce the center's rules automatically: a classroom cannot exceed its capacity, a student belongs to one classroom only, and students with attendance history cannot be deleted.
5. Give supervisors and admins reports such as attendance summaries, students who never received a gift and classrooms above the average size.
6. Build the project in a clean, well documented way using Scrum, GitHub and a layered architecture, so the team can deliver in three stages within 14 weeks.

1.4 Scope

The scope sets the limits of the project: what the system will do in this course and what it will not do, so the work stays focused and realistic.

Users of the system:

- Admin: manages students, staff assignments, classrooms, curriculum and the gift catalog.
- Teacher: takes attendance, records memorization progress and gives gifts in their own classroom.
- Supervisor: monitors the classrooms assigned to them and reads reports.

In scope (minimum viable product): login by role, student management, classroom and curriculum management, enrollment with a capacity check, daily attendance, memorization records, gifts, reports and protection against deleting students who have attendance records.

Nice to have (only if time allows): Dark mode and a read-only view for parents.

Out of scope: payments and fees, push notifications, audio recording of recitation, and web or desktop versions. Students do not log in to the app; they benefit from the records the staff keep.

1.5 Project Proposal

The following is a one-page summary of the proposal.

Table 1: Project proposal summary

Item	Description
Project	Noor Al-Iman, a Flutter mobile app for a Quran memorization center.
Problem	Attendance, memorization progress and gifts are recorded on paper. Progress is lost when a teacher is absent, attendance sheets go missing, some students never receive gifts, classrooms get over-filled and supervisors have no quick view of each classroom.
Solution	A mobile app for admins, teachers and supervisors, connected to a MySQL database through a secure PHP REST API, so every staff member sees the same up-to-date data.
Users	Admins, teachers and supervisors. Students do not log in.
Main features	Login by role; student management; classrooms, curriculum and enrollment with a capacity check; daily attendance; memorization records; gifts; reports; protection against deleting students with attendance history.
Out of scope	Payments, notifications, audio recording, web and desktop versions.
Approach	Scrum over 14 weeks in three stages (Plan & Design, Develop, Test & Deliver), managed with GitHub Issues, Projects and Milestones.
Technology	Flutter with Provider, PHP REST API, MySQL, GitHub Actions.
Expected result	A working Android app that replaces the paper notebooks, lets a teacher take attendance in under a minute and gives supervisors a live view of their classrooms.

Chapter 2: Methodology

2.1 Overview

Methodology is the planned way a team organizes its work from the first idea to the final delivery. Choosing it early matters, because it decides how requirements are collected, how work is split and how the team reacts when something changes [1].

2.2 Choosing a Development Approach

Before starting, we compared the three approaches we studied in previous courses.

Table 2: Comparison of development approaches

Approach	How it works	Fit for this project
Waterfall	Each phase (requirements, design, implementation, testing) is finished completely before the next one starts.	Weak. Feedback from the supervisor after each checkpoint would force us to go back to earlier phases, which is costly in Waterfall.
Extreme Programming (XP)	An Agile method built on engineering practices: very short releases, pair programming, test-driven development, continuous integration and an on-site customer.	Good practices, but it says little about planning in fixed stages, and writing tests before every feature is heavy for our timeline.
Scrum (Agile)	Work is done in short, fixed sprints. Each sprint ends with a working piece of the product that can be reviewed.	Best fit. It matches the course stages and checkpoints and lets us adjust after every review.

We chose **Scrum**, one of the most widely used Agile frameworks [2]. Scrum and XP are both Agile, but Scrum focuses on how the work is planned and reviewed, while XP focuses on engineering practices. We follow Scrum for planning and borrow two practices from XP: pair programming, where team members work through tasks together and review each other's code, and continuous integration, where every change pushed to GitHub is built and tested automatically using GitHub Actions.

The course is already divided into three stages with clear checkpoints, so each stage maps naturally to one or more sprints. Short sprints keep the work small enough to fit next to our other courses, and every sprint ends with something that can be shown and committed to GitHub, which also serves as proof of progress.

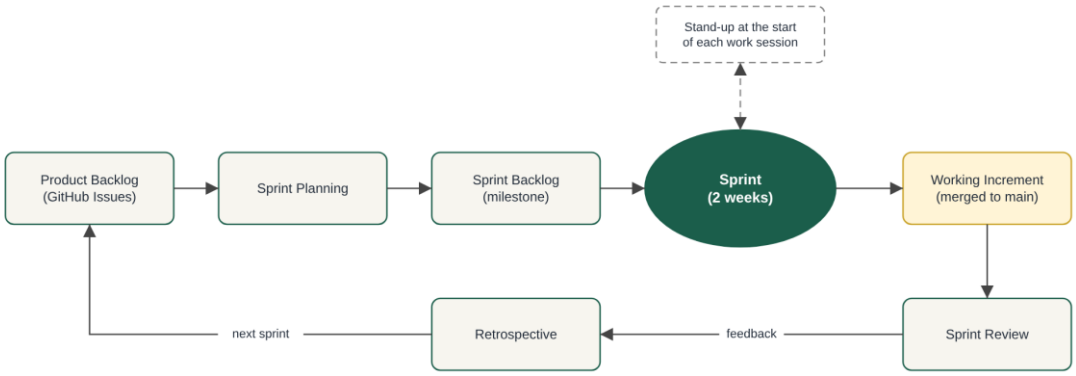


Figure 1: Scrum process followed by the team

2.3 Team Organization

Our team has three members. In Scrum there are three roles, and we filled them as follows:

Table 3: Scrum roles in the team

Role	Member	Responsibility
Product Owner	Laith Al Haddad	Keeps the product backlog in order, writes user stories and decides what is most important in each sprint.
Scrum Master	Rotates every sprint	Runs the short meetings, keeps the board up to date and helps remove anything that blocks the team.
Development Team	Adam Alrefa'e, Laith Al Haddad, Yousef Hassan	Design, build, review and test the product together.

2.4 Scrum Events

Table 4: Scrum events adapted to the team

Event	When	What we do
Sprint Planning	First day of each sprint (about 45 minutes)	Choose user stories from the backlog, split them into tasks and add them to the sprint milestone.
Stand-up	Start of every work session (5 minutes)	Each member says what was finished last time, what we do today and whether anything is blocking us.
Sprint Review	Last day of the sprint	Run the app together and check each story against its acceptance criteria.
Retrospective	Right after the review (15 minutes)	Agree on one thing that went well and one thing to improve in the next sprint.

2.5 Sprint Plan and Schedule

The plan follows the course schedule of 14 weeks in three stages. Stage 1 is one longer planning sprint. Stage 2 contains three development sprints of two weeks each and ends with a feature freeze, after which no new features are added. Stage 3 contains two sprints for quality, release and documentation.

Table 5: Sprint plan

Sprint	Weeks	Dates	Goal	Stage
Sprint 0	1-4	until 13 Nov 2026	Requirements, UI/UX design, architecture, repository and CI set-up	Stage 1: Plan & Design
Sprint 1	5-6	14 Nov to 27 Nov	Database and API base, login, Flutter skeleton and theme, dashboards, students API	Stage 2: Develop
Sprint 2	7-8	28 Nov to 11 Dec	Student and classroom screens, enrollment, attendance	Stage 2: Develop
Sprint 3	9-10	12 Dec to 25 Dec	Memorization, gifts, reports, delete rule (feature freeze)	Stage 2: Develop
Sprint 4	11-12	26 Dec to 8 Jan	Testing, bug fixing, performance and polish, release APK	Stage 3: Test & deliver
Sprint 5	13-14	9 Jan to 22 Jan 2027	README, user guide, demo video, final presentation	Stage 3: Test & deliver

A Gantt chart is a visual representation of tasks against time and helps a team track progress against its deadlines [3]. Figure 2 shows our plan with the three course checkpoints.

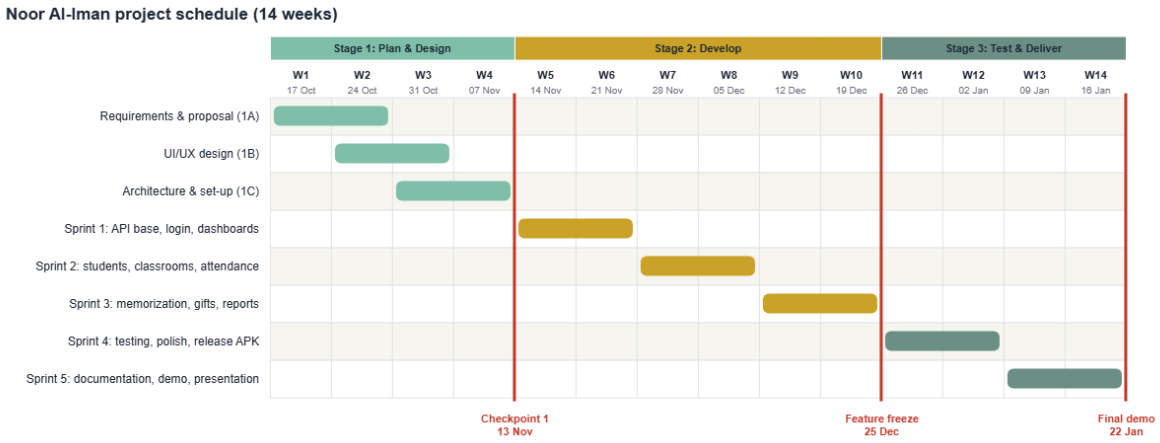


Figure 2: Gantt chart of the project schedule

2.6 Tools

- **GitHub Issues** for the product backlog. Each backlog task has an issue with the same number (task T007 is issue #7) and labels such as `app`, `api`, `docs`, `design`, `test` and `bug`.
- **GitHub Projects** as the sprint board with the columns To Do, In Progress, In Review and Done.
- **GitHub Milestones** for the three course stages, with the checkpoint dates as due dates.
- **Git branches and pull requests:** `main` holds stable work, `develop` collects finished tasks, and every task has its own branch merged through a reviewed pull request.

- **GitHub Actions** to check the code on every pull request and to build the Android APK automatically.
- **draw.io** for diagrams and **Figma** for the visual design.

2.7 Definition of Done

A task is moved to Done only when all of the following are true:

1. It meets the acceptance criteria of its user story.
2. It runs on an Android emulator or device without errors.
3. The automatic checks pass (analysis and tests).
4. A teammate reviewed and approved the pull request.
5. The related issue is closed and linked to the pull request.

Chapter 3: Requirements Analysis

3.1 Overview

This chapter covers part **1A** of the project. It describes who uses the system, what the system must do and how well it must do it, and it breaks the work into a backlog of tasks. The one-page proposal summary is given in Section 1.5.

3.2 Requirements Gathering

No single technique can capture every requirement, so a combination of methods gives better coverage [4]. We used three techniques:

1. **Interviews:** we held semi-structured interviews with a teacher and a supervisor at the center. We used a short list of open questions about their daily routine, the records they keep and the problems they face, and asked follow-up questions where needed. These interviews showed that attendance and progress tracking take the most time and cause the most mistakes.
2. **Observation:** we used interactive observation, attending a class and asking the teacher questions while they worked. We watched how attendance is called out and how progress is written in the notebooks. This shaped the design of the attendance screen, which must work quickly while the teacher is standing in front of the class.
3. **Document analysis:** we reviewed the paper attendance sheets and progress notebooks to understand exactly which information is recorded and how.

3.3 Stakeholders and Users

Table 6: Stakeholders of the system

Stakeholder	Uses the app	Main needs
Admin	Yes	Register students and staff, create classrooms and curriculum, manage the gift catalog, read reports.
Teacher	Yes	See their classroom, take attendance quickly, record memorization, give gifts.
Supervisor	Yes	Monitor assigned classrooms and read attendance and progress reports.
Student	No	Benefits from accurate records and fair rewards.
Parent / guardian	Future work	A read-only view of their child's progress.

3.4 Use Case Model

Actors are the people or systems that interact with the system, and use cases are the goals they achieve with it [5]. The system has three actors: Admin, Teacher and Supervisor. All three share common use cases such as logging in, so they are shown as specializations of a general Staff User actor. Figure 3 shows the complete use case model.

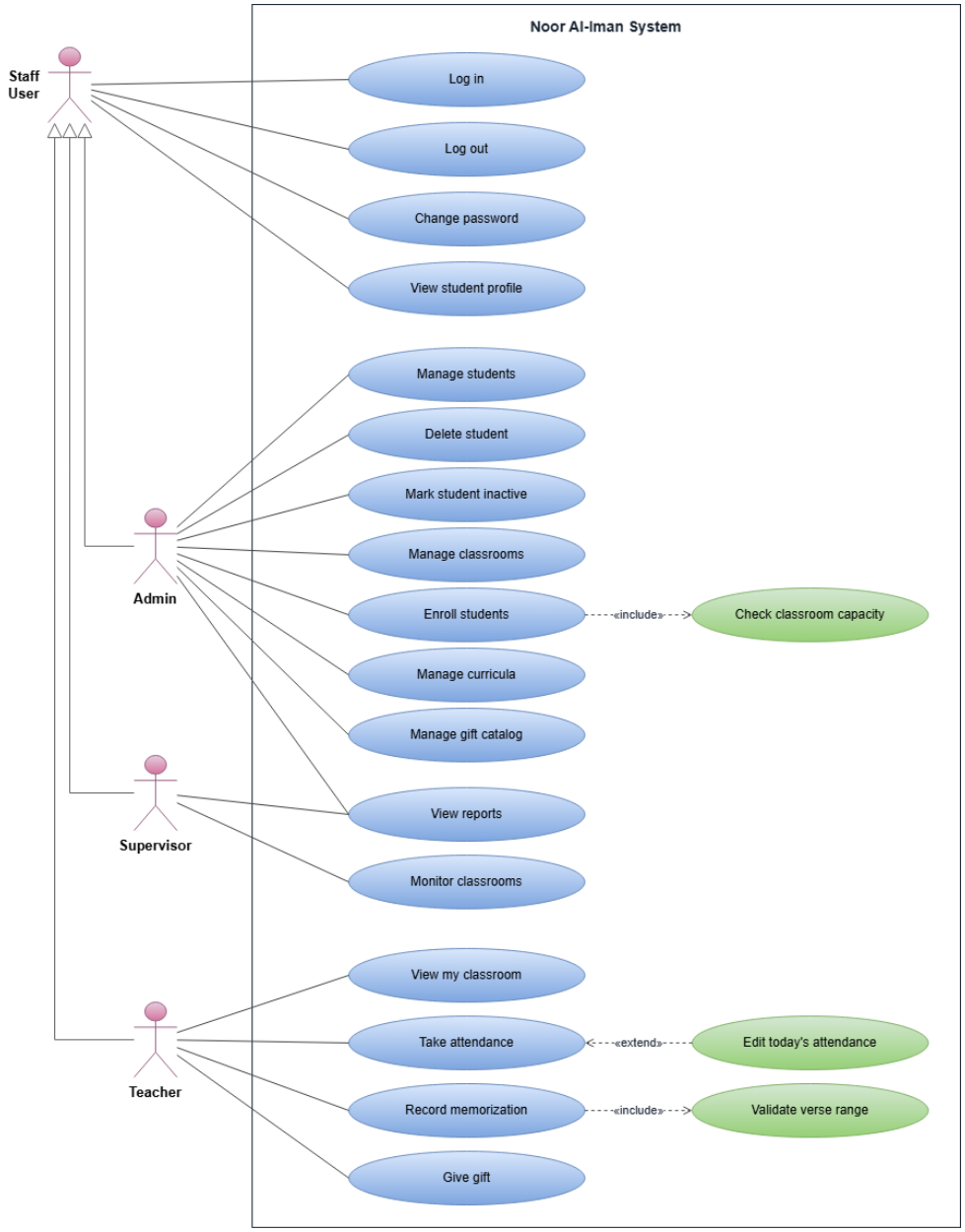


Figure 3: Use case diagram

Two use cases are always performed as part of another one: checking the classroom capacity is included in enrolling students, and validating the verse range is included in recording memorization. Editing today's attendance extends taking attendance: it only happens when attendance for that date was already saved. Marking a student inactive is a separate admin use case, used when a student leaves the center or cannot be deleted because they have attendance records.

3.5 User Stories

Each user story describes one need from the point of view of a user, followed by acceptance criteria that tell us when the story is complete. The story IDs are reused in GitHub so a story can be traced from this report to the code.

Table 7: User stories

ID	User story	Acceptance criteria
US-001	As a staff member, I want to log in with my email, password and role so that I only see the functions of my role.	A wrong login shows an error. A correct login opens the dashboard of the chosen role.
US-002	As a staff member, I want to log out so that nobody else can use my account on a shared phone.	Logging out returns to the login screen and clears the saved session.
US-003	As a staff member, I want to change my password so that I can keep my account secure.	The old password is required. The new password must have at least 6 characters.
US-004	As an admin, I want to register a new student with their personal details so that they can join a classroom.	First name, last name and zip code are required. The city is filled in from the zip code.
US-005	As an admin, I want to edit a student's details so that their information stays correct.	Changes are saved and appear on the profile right away.
US-006	As an admin, I want to search students by name or ID so that I can find a student quickly.	Results update while typing and can be filtered by classroom.
US-007	As an admin, I want to mark a student as inactive when they leave the center so that the lists stay clean without losing their history.	Inactive students are hidden from lists by default and can be activated again.
US-008	As an admin, I want to create a classroom with a teacher, a supervisor, a curriculum and a maximum size.	All four fields are required and the classroom appears in the list.
US-009	As an admin, I want to enroll students in a classroom without going over its maximum size.	Enrollment is refused with a clear message when the classroom is full.
US-010	As an admin, I want to manage the curriculum and the gift catalog so that teachers can choose from up-to-date lists.	New curriculum and gift types can be added and appear in the selection lists.
US-011	As a teacher, I want to see my classroom and its students when I open the app.	The home screen shows the classroom, the curriculum, the number of students and the student list.
US-012	As a teacher, I want to mark attendance for the whole classroom in one screen.	Each student can be marked Present, Absent or Late with an optional note, and one save stores the whole class.
US-013	As a teacher, I want to correct today's attendance if I made a mistake.	Opening the same date again shows the saved marks, which can be changed and saved again.
US-014	As a teacher, I want to record which Surah and verses a student memorized and how well they recited.	Surah, from verse, to verse and date are required, and the verse range must fit the chosen Surah.
US-015	As a teacher, I want to give a gift to a student so that their effort is rewarded and recorded.	The gift appears in the student's gift history with the date and the teacher's name.
US-016	As a teacher or supervisor, I want to open a student's profile to see their progress, attendance and gifts in one place.	The profile shows summary numbers and three tabs: Progress, Attendance and Gifts.

ID	User story	Acceptance criteria
US-017	As a supervisor, I want to monitor the classrooms I am responsible for.	Each assigned classroom shows its teacher, student count and attendance rate for the week.
US-018	As a supervisor or admin, I want an attendance summary for a chosen period.	The report lists present, absent and late counts per student for the selected period.
US-019	As a supervisor or admin, I want to see students who never received a gift so that nobody is overlooked.	The report lists every active student without a gift record.
US-020	As an admin, I want the system to stop me from deleting a student who has attendance records.	A dialog explains why and offers to mark the student inactive instead.

3.6 Functional Requirements

Functional requirements describe what the system must do: the features and behavior users interact with. The table below lists the 22 functional requirements and the user stories they come from.

Table 8: Functional requirements

ID	Requirement	Story
FR-001	The system shall authenticate staff by email, password and role, and issue a session token that expires after 12 hours.	US-001
FR-002	The system shall show each role only its own dashboard, navigation items and actions.	US-001
FR-003	The system shall allow a user to log out and shall return to the login screen when the session expires.	US-002
FR-004	The system shall allow a user to change their password after confirming the old one.	US-003
FR-005	The system shall allow the admin to create a student record with name, birth date, gender, phone, address and zip code.	US-004
FR-006	The system shall fill in the city automatically from the entered zip code.	US-004
FR-007	The system shall allow the admin to update a student's details and classroom.	US-005
FR-008	The system shall allow searching students by name or ID and filtering them by classroom.	US-006
FR-009	The system shall allow the admin to mark a student as inactive or active again.	US-007
FR-010	The system shall refuse to delete a student who has attendance records.	US-020
FR-011	The system shall allow the admin to create and edit classrooms with a name, maximum size, teacher, supervisor and curriculum.	US-008
FR-012	The system shall allow the admin to add curriculum and gift types.	US-010
FR-013	The system shall refuse an enrollment that would exceed the classroom's maximum size.	US-009

ID	Requirement	Story
FR-014	The system shall keep each student in exactly one classroom at a time.	US-009
FR-015	The system shall show a teacher their classroom with each student's last memorized Surah and today's attendance status.	US-011
FR-016	The system shall record attendance for a whole classroom on a selected date with a status and an optional note for each student.	US-012
FR-017	The system shall keep only one attendance record per student per day and update it when the same date is saved again.	US-013
FR-018	The system shall record memorization sessions with Surah, verse range, date, evaluation and notes, and reject a verse range outside the chosen Surah.	US-014
FR-019	The system shall record each gift with its type, the student who received it, the teacher who gave it and the date.	US-015
FR-020	The system shall display a student profile with summary numbers and the history of memorization, attendance and gifts.	US-016
FR-021	The system shall produce reports for attendance per student, students without gifts, classrooms above the average size and the number of memorization records in a period.	US-017, US-018, US-019
FR-022	The system shall limit teachers and supervisors to the data of their own classrooms, and check this on the server for every request.	US-011, US-017

3.7 Non-Functional Requirements

Non-functional requirements describe how well the system must work, such as its speed, security and ease of use.

Table 9: Non-functional requirements

ID	Category	Requirement
NFR-001	Usability	A teacher shall be able to take attendance for 20 students in under one minute. All touch targets shall be at least 48 by 48 dp.
NFR-002	Performance	Common screens shall load in under 2 seconds on a normal 4G connection with up to 500 students in the database.
NFR-003	Security	All traffic shall use HTTPS. Passwords shall be stored as bcrypt hashes. The app shall never contain database credentials, and all SQL shall use prepared statements.
NFR-004	Reliability	When the connection fails, the app shall show a clear message with a Retry option instead of crashing. Important rules shall also be enforced by the database itself.
NFR-005	Maintainability	The code shall follow a layered structure with one state management approach, and every change shall pass automatic analysis and tests before merging.
NFR-006	Portability	The app shall run on Android phones with Android 8.0 or newer, in portrait mode, and on Windows 10 or newer as a desktop app. The server shall run on PHP 8.2.35 or newer with MySQL 8.

ID	Category	Requirement
NFR-007	Accessibility	Text colours shall meet the WCAG 2.1 AA contrast ratio, and status shall never be shown by colour alone [7].
NFR-008	Localization	Layouts shall be ready for Arabic and right-to-left display without redesign.

3.8 Feature List and Scope

Table 10: Feature list

#	Feature	Scope
F001	Login, logout and role-based dashboards	Minimum viable product
F002	Student management (add, edit, search, profile, inactive)	Minimum viable product
F003	Classroom and curriculum management	Minimum viable product
F004	Enrollment with a capacity check	Minimum viable product
F005	Daily attendance	Minimum viable product
F006	Memorization records	Minimum viable product
F007	Gift catalog and giving gifts	Minimum viable product
F008	Reports	Minimum viable product
F009	Delete protection for students	Minimum viable product
F010	Arabic language and right-to-left layout	Minimum viable product
F011	Dark mode	Nice to have
F012	Parent read-only view	Nice to have
F013	Payments, notifications, audio recording, web version	Out of scope

3.9 Assumptions and Constraints

- The center has internet access, and a VPS is available to host the API and the MySQL database.
- The app is used by one center, so multi-branch support is not needed.
- Admins create staff accounts. There is no public sign-up.
- The list of 114 Surahs with their verse counts is fixed and stored inside the app and the API.
- The team has 14 weeks (until 22 January 2027), and all features must be finished by the feature freeze on 25 December 2026.

3.10 Task Backlog

The backlog breaks the project into tasks. Each task is estimated in story points (1, 2, 3, 5 or 8, where 8 is a large screen with logic) and assigned to a sprint. Each task becomes a GitHub issue with the same number.

Table 11: Task backlog

Task	Description	Sprint	Points
T001	Problem statement, user stories and scope	0	3
T002	Task backlog and GitHub Projects board	0	2
T003	Wireframes for all screens	0	5
T004	Navigation flow, colours and typography	0	3
T005	Repository set-up, folder structure, CI, state management choice	0	3
T006	Data model and API design	0	3
T007	Create MySQL database (schema and demo data)	1	3
T008	PHP API skeleton: routing, database connection, JSON errors	1	5
T009	Login endpoint with tokens and role checks	1	5
T010	Flutter project, theme, routes and API client	1	5
T011	Login screen and role dashboards	1	5
T012	Students API	1	5
T013	Student list, form and profile screens	2	8
T014	Classrooms API with enrollment and capacity check	2	5
T015	Classroom list, details and form screens	2	5
T016	Attendance API	2	3
T017	Take attendance screen	2	5
T018	Memorization API and Add Progress screen	3	5
T019	Progress and attendance history on the profile	3	3
T020	Gifts API	3	3
T021	Gift catalog and Give Gift screens	3	3
T022	Reports API	3	3
T023	Reports screen with chart	3	5
T024	Delete protection and mark inactive	3	2
T025	Validation and error states	4	3
T026	Unit, widget and API tests, test report	4	5
T027	Deploy on the server and release the APK	4	3
T028	README, architecture diagram and user guide	5	3
T029	Arabic translation and right-to-left layout (optional)	4	5
T030	Performance, accessibility, app icon and splash	4	3
T031	Demo video, final presentation and viva preparation	5	3

The backlog totals 122 story points. The three development sprints hold 78 points, about 26 per sprint, while Stage 3 is kept lighter on purpose because testing and fixing usually take longer than planned. The scope is realistic for three reasons: the data model is designed before development starts, most screens are list, form or detail screens built from the same components, and the nice to have features can be dropped without affecting the minimum viable product.

Chapter 4: UI/UX Design

4.1 Overview

This chapter covers part **1B**. It lists every screen of the app, shows how users move between them, presents the design of each screen and defines the colours, typography and components used across the app. The screens were designed from the user stories in Chapter 3, and each one lists the data it needs so the person building it knows where its data comes from.

4.2 Design Goals

- **Fast for teachers.** Teachers use the app in the middle of a class, so common actions such as attendance must take a few taps, not long forms.
- **Clear for supervisors.** Supervisors mostly read, so their screens use numbers and progress bars that can be understood briefly.
- **Consistent.** Lists, cards, forms and buttons look and behave the same on every screen.
- **Calm and respectful.** The center is a Quran school, so the style uses a deep green, a small touch of gold and plenty of white space.
- **Ready for Arabic.** Layouts avoid fixed left and right positions so they can be mirrored for right-to-left text later.

4.3 Screen Inventory

Table 12: Screen inventory

ID	Screen	Used by	Main data
S001	Splash	All	None
S002	Login	All	Person, Admin, Teacher, Supervisor
S003	Admin Dashboard	Admin	Student, Classroom, Attendance_Record
S004	Teacher Home (My Classroom)	Teacher	Classroom, EnrollsIn, Memorization_Record, Attendance_Record
S005	Supervisor Dashboard	Supervisor	Classroom, Attendance_Record
S006	Students List	Admin (Teacher read only)	Student, Person, EnrollsIn
S007	Add / Edit Student	Admin	Person, Student, ZipCodes, EnrollsIn
S008	Student Profile	All	Person, Student, Memorization_Record, Attendance_Record, TakenGift
S009	Classrooms List	Admin, Supervisor	Classroom, Curriculum, EnrollsIn
S010	Classroom Details	Admin, Supervisor	Classroom, EnrollsIn, Person
S011	New / Edit Classroom	Admin	Classroom, Teacher, Supervisor, Curriculum

ID	Screen	Used by	Main data
S012	Take Attendance	Teacher	Attendance_Record
S013	Add Progress	Teacher	Memorization_Record
S014	Give Gift	Teacher	GiftType, Gift, GiveGift, TakenGift
S015	Gift Catalog	Admin	GiftType, Gift
S016	Reports	Admin, Supervisor	Attendance_Record, TakenGift, EnrollsIn, Memorization_Record
S017	Profile & Settings	All	Person
S018	Delete Check Dialog	Admin	Student, Attendance_Record
S019	Curriculum (list with an add dialog)	Admin	Curriculum

Some actions open a dialog or a bottom sheet inside an existing screen instead of a new screen: enrolling students opens a bottom sheet in S010, adding a gift type opens a dialog in S015, and marking a student inactive opens a confirmation dialog from S008. After confirming, the user returns to the student profile, which then shows the Inactive status.

4.4 Navigation Flow

Figure 4 shows how users move between screens. After the splash and login screens, each role opens its own dashboard. List screens lead to detail screens and detail screens lead to forms and dialogs. Profile and Settings can be opened from the bottom navigation bar on every main screen, and logging out always returns to the login screen. Dialogs and bottom sheets are shown in yellow.

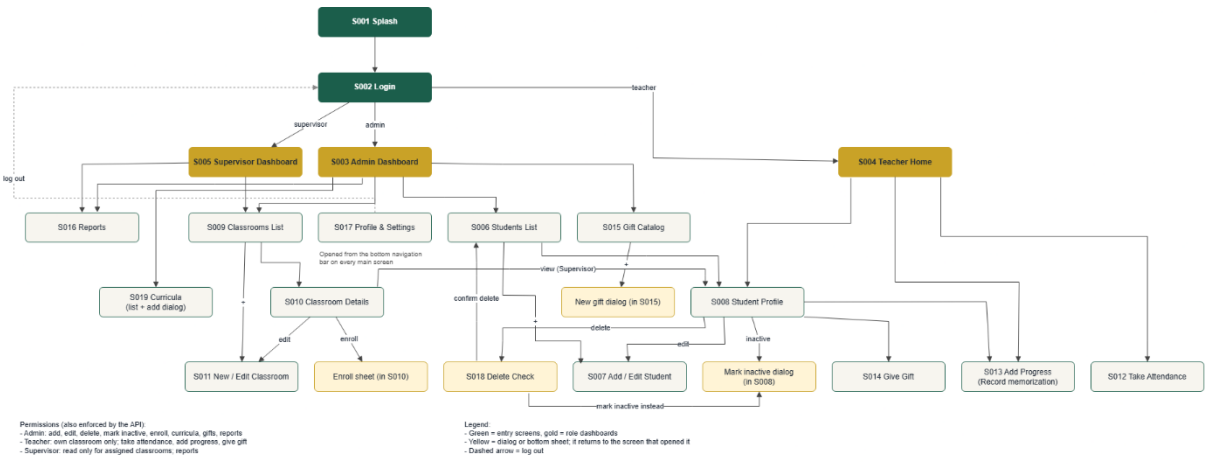


Figure 4: Navigation flow between screens

Table 13: Bottom navigation items by role

Tab	Admin	Teacher	Supervisor
Home	Admin Dashboard	My Classroom	Supervision
Students	All students	Students of my classroom	Students of my classrooms

Tab	Admin	Teacher	Supervisor
Classes	All classrooms	Hidden	My classrooms
Reports	All reports	Hidden	Reports for my classrooms
Profile	Yes	Yes	Yes

4.5 Screen Designs

We first sketched each screen as a low-fidelity wireframe to agree on layout and content, then produced the final high-fidelity designs shown here. Every screen was designed in a light and a dark theme, and in English and Arabic with a mirrored right-to-left layout. The figures below show the main state of each screen in the light English version. Each screen also has designs for its other states, such as loading, empty lists, validation errors and lost connection.

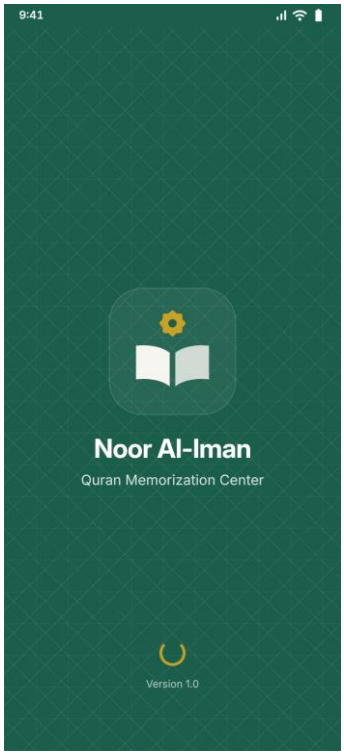


Figure 5: S001 Splash

Shows the logo while the app checks for a saved login token and asks the API if it is still valid. It moves to Login or to the role dashboard.

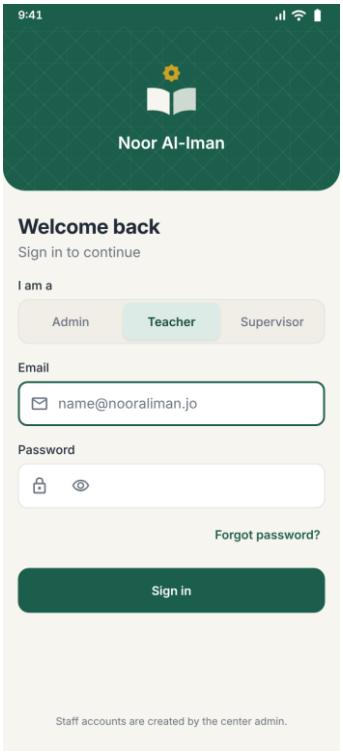


Figure 6: S002 Login

The user picks a role (Admin, Teacher, Supervisor), then enters email and password. The password field has a show or hides icon. A wrong login shows a clear error under the button.



Figure 7: S003 Admin Dashboard

A greeting, three quick numbers (students, classrooms, attendance rate this month) and six big tiles that open each management area. The bottom navigation bar is shared by all main screens.

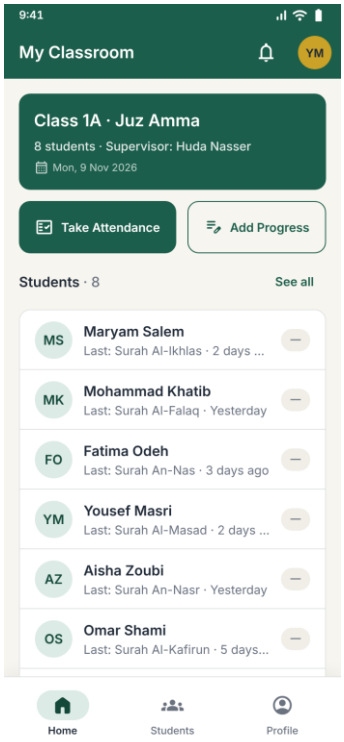


Figure 8: S004 Teacher Home (My Classroom)

The teacher sees today's class card and two main actions: Take Attendance and Add Progress. Below is the student list, showing each student's last memorized Surah and a badge for today's attendance, which shows a dash until attendance is taken.

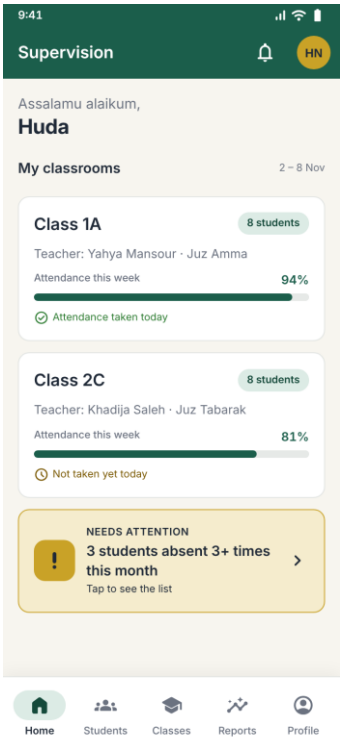


Figure 9: S005 Supervisor Dashboard

One card for each classroom the supervisor monitors, showing the teacher, student count and a weekly attendance bar. A 'Needs attention' card lists students with many absences.



Figure 10: S006 Students List

Search by name or ID and filter by classroom with chips. Tapping a row opens the profile. The plus button opens the Add Student form.

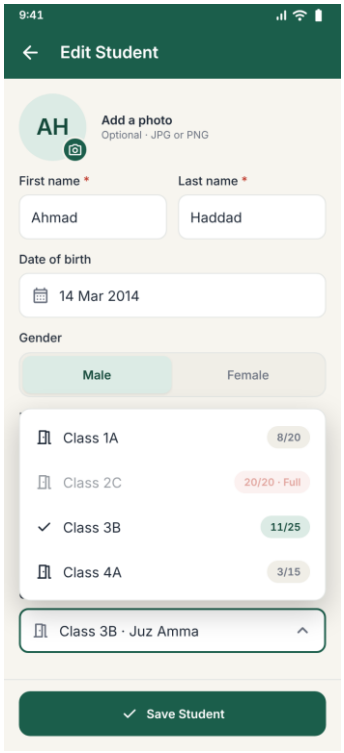


Figure 11: S007 Add / Edit Student

One form used for both adding and editing. Entering a zip code fills in the city from the ZipCodes table, so the city is never typed by hand. Required fields are checked before saving.

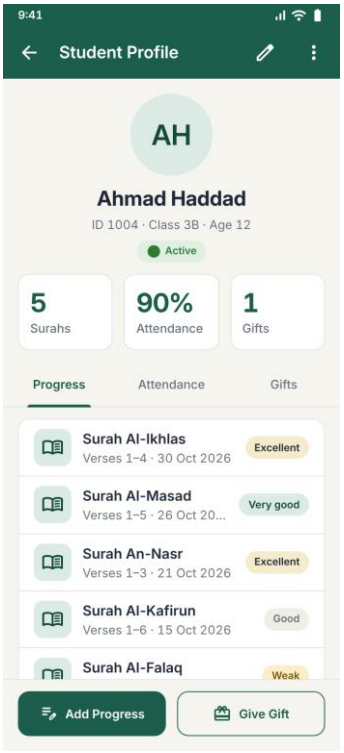


Figure 12: S008 Student Profile

Header with basic info and three summary numbers. Three tabs: Progress (memorization history), Attendance and Gifts. Buttons for Edit and Add Progress. The top right menu has Delete.

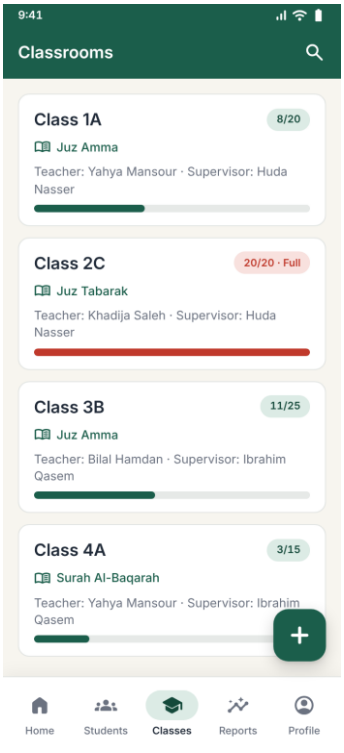


Figure 13: S009 Classrooms List

Each card shows the class, its curriculum, teacher, supervisor and a capacity bar (enrolled / MaxSize). Full classes are easy to spot.

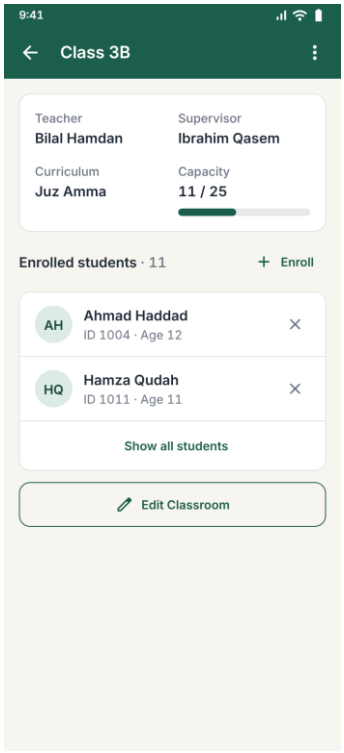


Figure 14: S010 Classroom Details

Shows the teacher, supervisor, curriculum and capacity. Lists enrolled students with a remove button. '+ Enroll' opens a picker that refuses new students when the class is full.

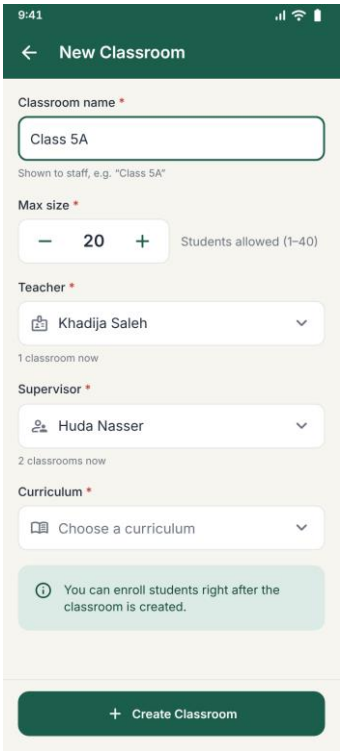


Figure 15: S011 New / Edit Classroom

Form with name, max size and three drop-downs filled from the Teacher, Supervisor and Curriculum tables.

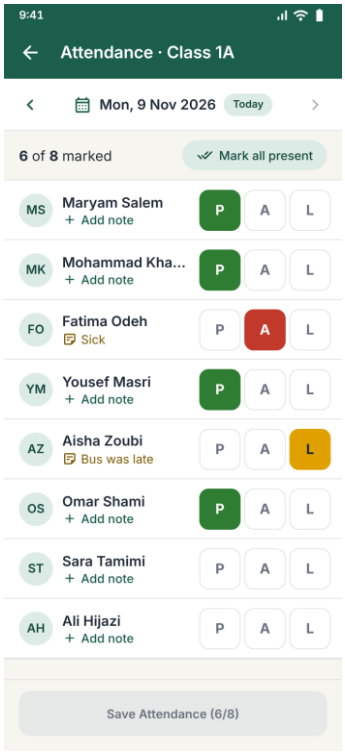


Figure 16: S012 Take Attendance

Date at the top, then each student with P / A / L toggles and an optional note. 'Mark all present' saves time. The save button shows how many students are marked.

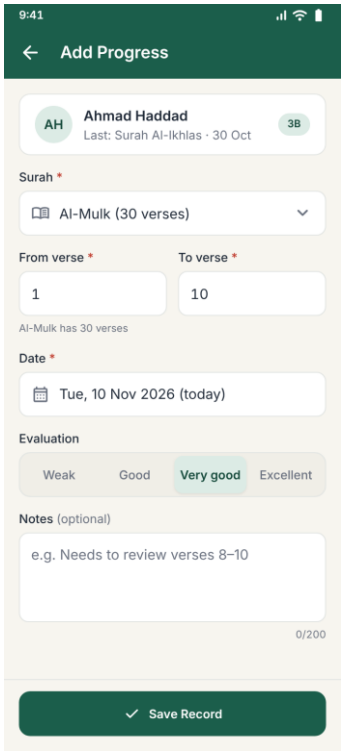


Figure 17: S013 Add Progress

Pick the Surah from a list of 114, enter the verse range and date, and choose an evaluation. The app checks that the verse range fits inside the chosen Surah.

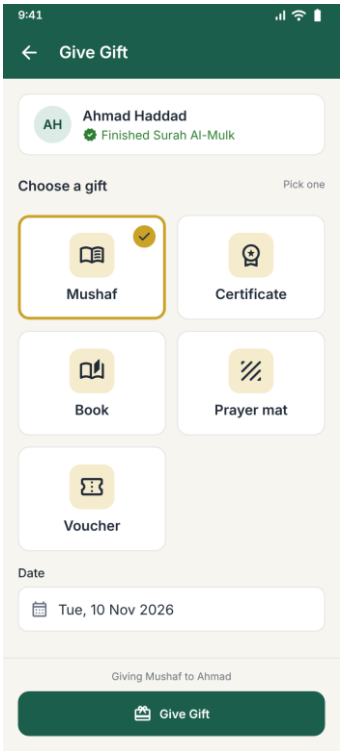


Figure 18: S014 Give Gift

Choose one gift from a grid and confirm. The API creates one Gift record, then GiveGift (which teacher gave it) and TakenGift (which student received it) in one transaction.

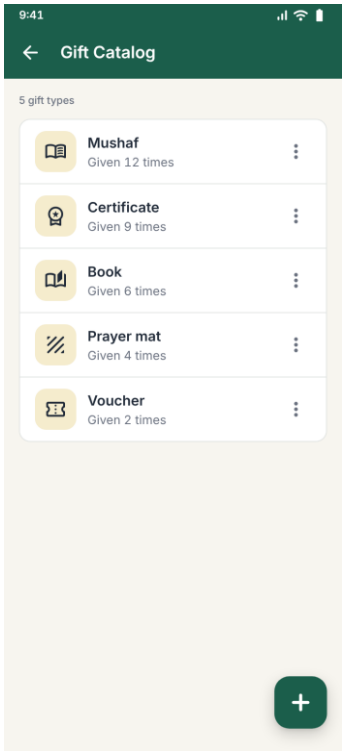


Figure 19: S015 Gift Catalog

List of gift types and how many times each was given. The plus button adds a new gift type.

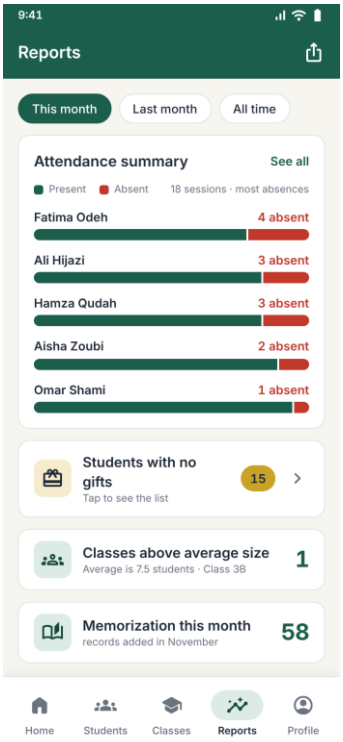


Figure 20: S016 Reports

Period filter, then report cards: attendance summary chart, students with no gifts, classes above average size and the number of memorization records.

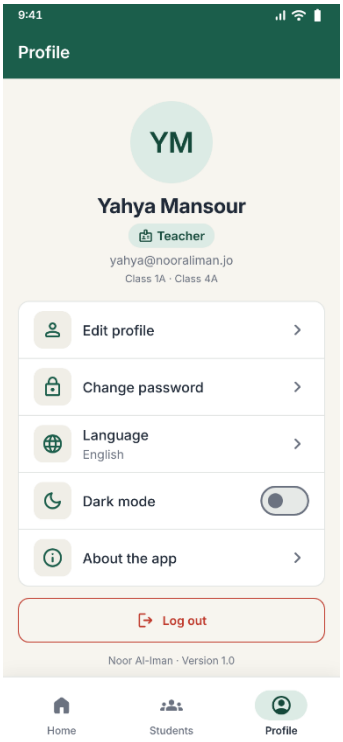


Figure 21: S017 Profile & Settings

The logged in user's info, change password, language, dark mode, about, and log out.

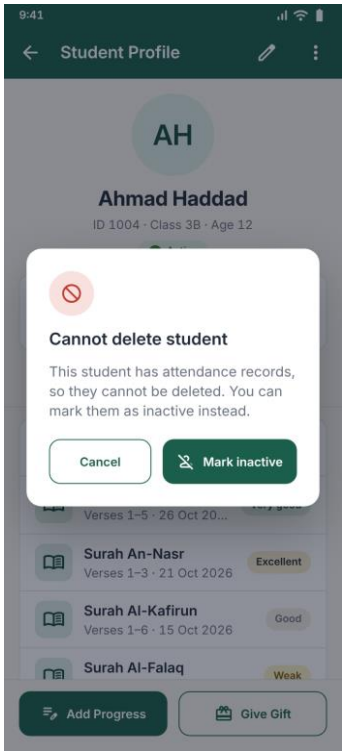


Figure 22: S018 Delete Check Dialog

Shown when an admin tries to delete a student who has attendance records. The API refuses the delete (the database blocks it too), and the app offers 'Mark inactive' instead.

The figure below shows some of the other states that were designed, so users always get clear feedback when something goes wrong.

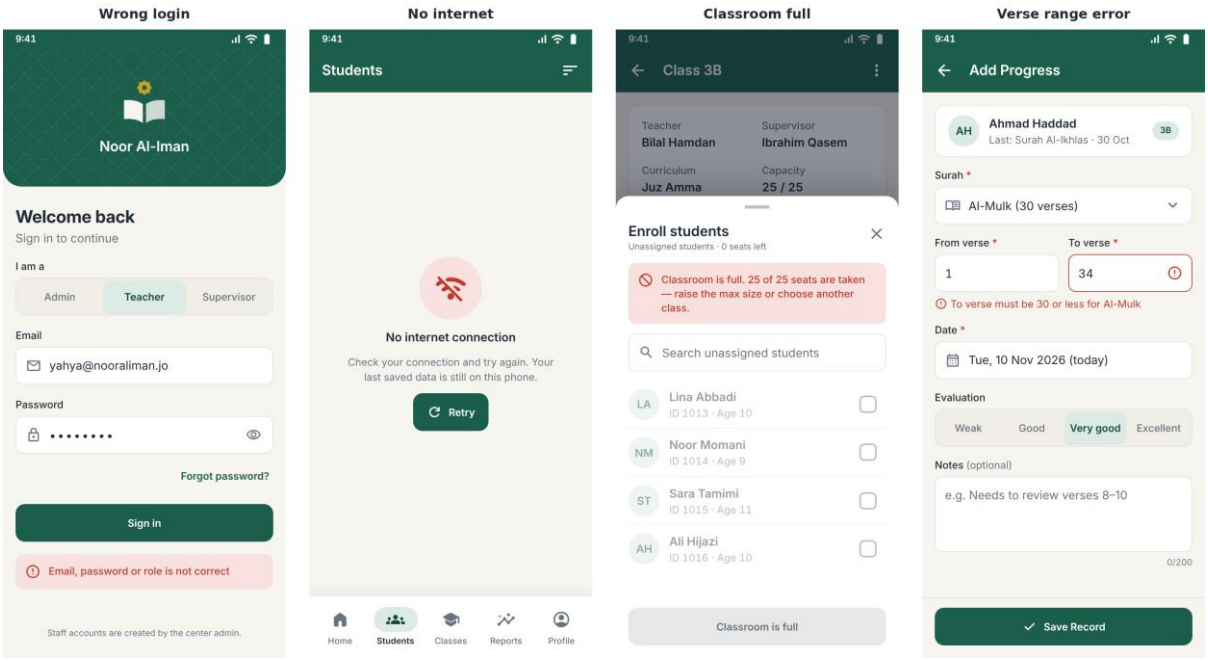


Figure 23: Examples of error and empty states

The next figure shows the same screen in the four versions of the design. Supporting Arabic from the design stage means the layout can be mirrored without redesign.

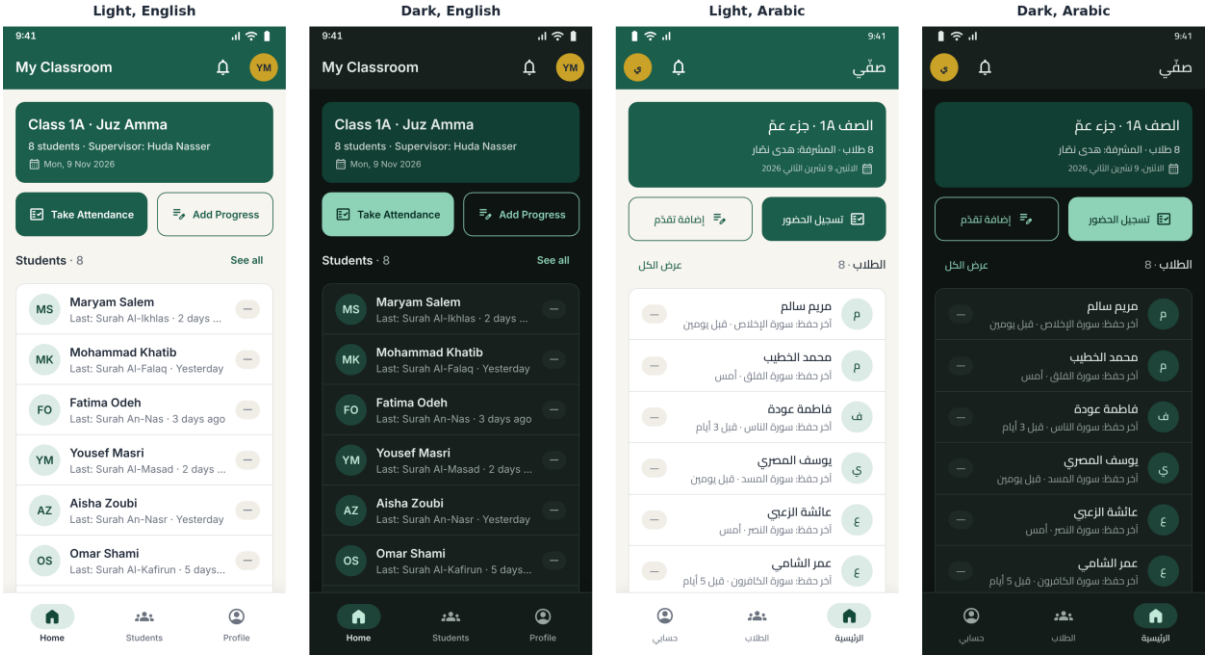


Figure 24: Teacher Home in light, dark, English and Arabic

4.6 Colours

The palette suits a Quran center: a deep green as the main colour, a warm gold for highlights and an ivory background that is softer on the eyes than pure white. Status colours are used only for attendance and messages, so they always mean the same thing.

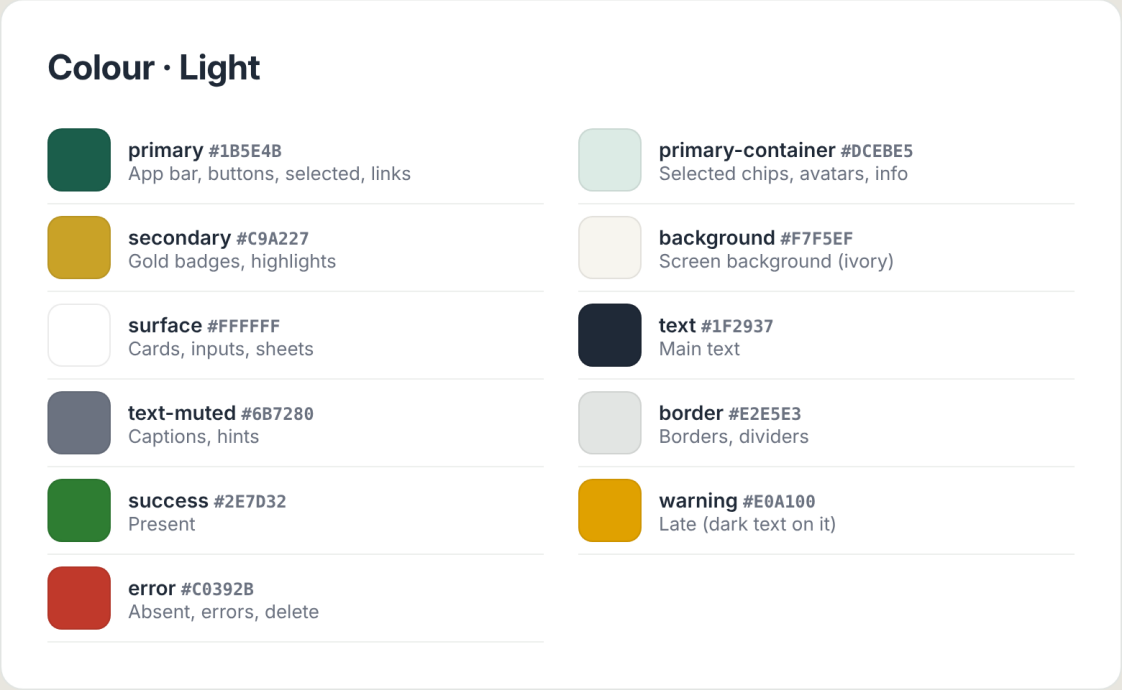


Figure 25: Colour palette (light theme)

Table 14: Colour usage

Name	Hex	Used for
Primary (Deep Green)	#1B5E4B	App bar, main buttons, selected tab, links
Primary container	#DCEBE5	Selected chips, avatars, info cards
Secondary (Gold)	#C9A227	Highlights, badges, selected gift
Background (Ivory)	#F7F5EF	Screen background
Surface (White)	#FFFFFF	Cards, inputs, dialogs
Text (Charcoal)	#1F2937	Body text and titles
Text muted	#6B7280	Captions and hints
Border	#E2E5E3	Borders and dividers
Success	#2E7D32	Present status, success messages
Warning	#E0A100	Late status, warnings
Error	#C0392B	Absent status, errors, delete actions

We checked the main text colours against the WCAG 2.1 contrast guideline [7]. White on deep green reaches 7.6:1 and charcoal on ivory reaches 13.5:1, both well above the 4.5:1 minimum. White on gold only reaches 2.4:1, so text on gold is always charcoal. Attendance status is also never shown by colour alone: every chip shows the word or the letter (P, A or L).

4.7 Typography

The app uses **Inter** for English text and **Cairo** for Arabic text. Both are free Google fonts that are easy to read on small screens, and Cairo was designed to sit well next to Latin fonts, so switching languages does not change the feel of the app.



Figure 26: Type scale in English and Arabic

Table 15: Type scale

Style	Size	Weight	Used for
Display	28 sp	Bold	Splash title, large numbers on dashboards
Headline	22 sp	Bold	Greetings, profile name
Title	18 sp	SemiBold	App bar and card titles
Body	15 sp	Regular	Normal text, list items
Label	13 sp	Medium	Buttons, chips, field labels
Caption	11 sp	Regular	Dates, hints, secondary details

4.8 Components and Usability Decisions

All screens are built from a small set of shared components: buttons (primary, outlined and danger), outlined text fields with the label above, status chips, person tiles with an avatar, stat cards, progress bars, empty states, error states with a Retry button, confirmation dialogs and a bottom navigation bar. Spacing follows an 8 point grid, cards and buttons use 12 point rounded corners, and icons come from the Material Symbols set.

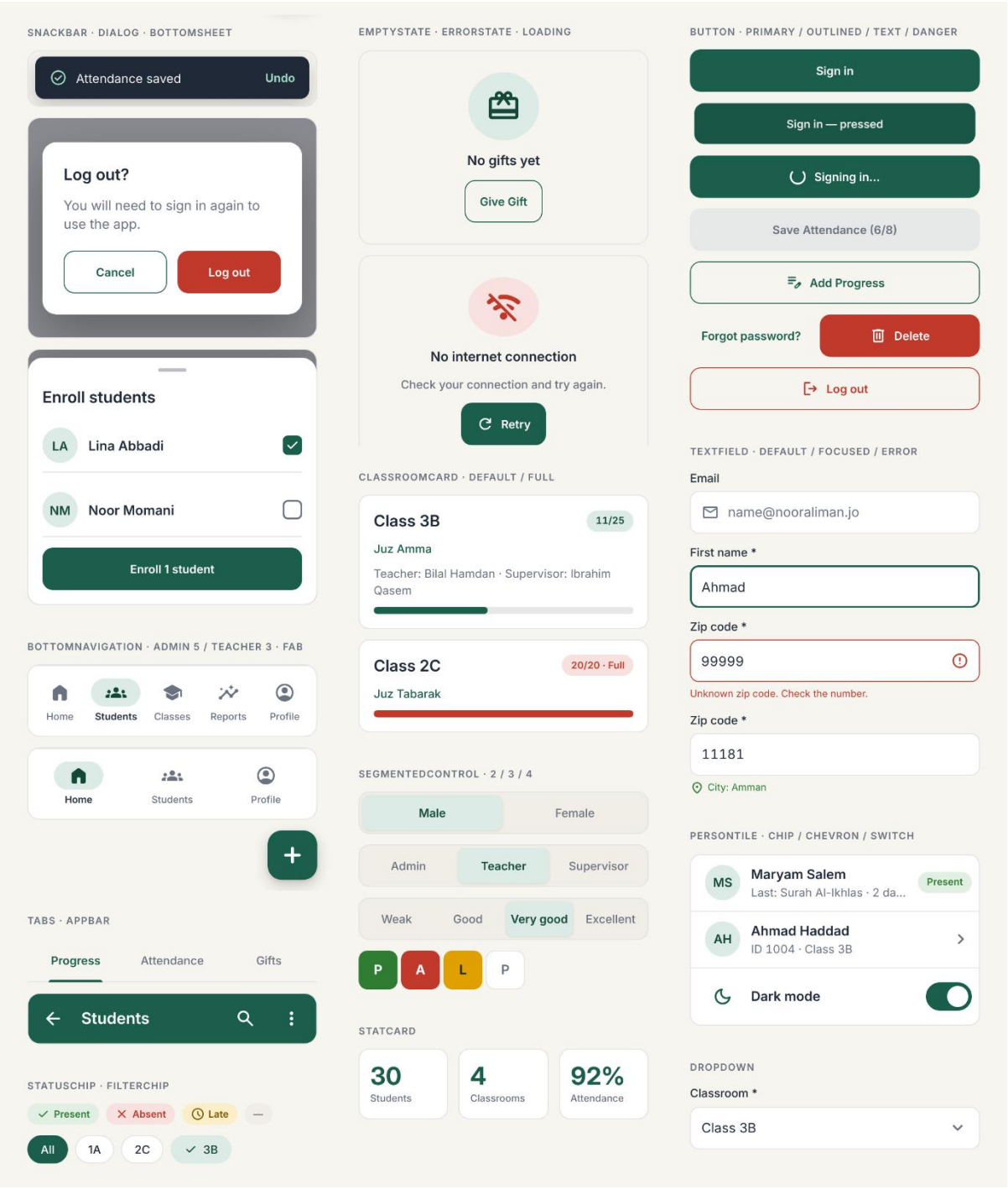


Figure 27: Shared components

Several usability decisions come directly from our observation of the center. Attendance is taken for the whole class in one screen with a single tap per student, and a "Mark all present" button saves time because most students are present on most days. Typing the zip code fills in the city automatically. Mistakes are prevented rather than explained: a full classroom cannot accept new students and the verse range is checked against the chosen Surah. Deleting always asks for confirmation, and students with attendance can only be marked inactive.

Chapter 5: Architecture and Project Set-up

5.1 Overview

This chapter covers part **1C**. It describes the overall architecture of the system, the technology chosen for each part, the state management approach, the data model, and how the GitHub repository and automated builds are set up.

5.2 System Architecture

The system follows a client-server architecture. The Flutter app on the staff phone never talks to the database directly. Instead, it sends HTTPS requests to a REST API written in PHP, which runs on our VPS next to the MySQL database. This keeps the database private, keeps database credentials out of the app and lets the server check every request against the user's role. The figure below shows how the parts connect, including how the APK reaches the staff phones through GitHub.

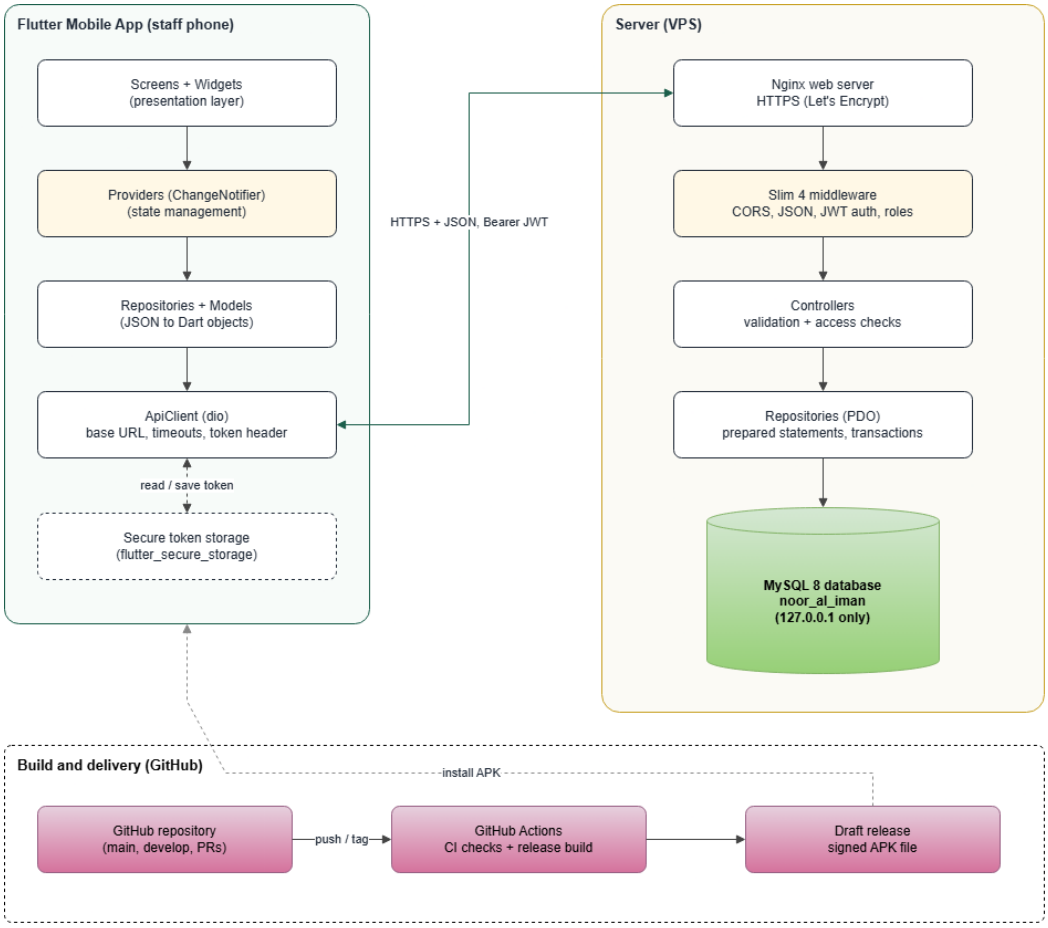


Figure 28: System architecture and data connection

Login returns a JWT token that expires after 12 hours and is kept in secure storage on the phone. Logging out deletes the token from the phone. The token is not revoked on the server, so a copied token would stay valid until it expires; we accept this as a known limitation of the current design.

Both sides are split into layers, and each layer only talks to the layer directly below it. On the phone, screens read data from providers, providers call repositories, and repositories call the API client. On the server, middleware checks the request and the token, controllers validate the input and the user's access, and repositories apply the business rules (such as the capacity check) and run the SQL. This separation keeps networking code out of the screens and SQL in one place on the server.

5.3 Technology Stack

Table 16: Technology stack

Part	Technology	Reason
Mobile app	Flutter and Dart 3	Course requirement, one code base for Android and iOS.
State management	Provider	Explained in Section 5.4.
Navigation	go_router	Named routes and simple redirects for role checks.
HTTP client	dio	One place to add the login token and handle errors.
Token storage	flutter_secure_storage	Keeps the login token in the Android Keystore.
Backend	PHP 8.2 with the Slim 4 framework	Light framework with clear routing and middleware, easy to host on our VPS.
Database	MySQL 8	Relational data with foreign keys, constraints, triggers and views.
Authentication	JWT tokens and bcrypt password hashing	Stateless sessions that expire, no plain-text passwords.
Web server	Nginx with HTTPS (Let's Encrypt)	Standard, fast and free certificates.
Automation	GitHub Actions	Checks every pull request and builds the APK.

We also compared other ways of storing the data. Keeping a local database on each phone would mean every phone has different data, which does not work for a shared center. Firebase offers a free cloud database, but it is a document database without joins or foreign keys, so rules such as classroom capacity and the reports would be harder to implement correctly. Connecting the app directly to MySQL would place the database password inside the app, where anyone could extract it. A central MySQL database behind our own API avoids all of these problems.

5.4 State Management

State management is how the app keeps track of data that changes over time, such as a list of students loaded from the server, and how it updates the screens when that data changes. Flutter supports several approaches [6]. We compared the most common ones:

Table 17: Comparison of state management approaches

Approach	Advantages	Disadvantages	Decision
setState	Built into Flutter, nothing extra to learn.	State cannot be shared between screens, and logic gets mixed with the interface.	Too limited
Provider	Recommended by the Flutter documentation for apps of this size, simple and covered in the course.	Less strict than BLoC, so the team needs its own rules.	Chosen
Riverpod	Safer at compile time and does not depend on BuildContext.	More concepts and code generation to learn.	Good but heavier
BLoC	Very structured, suited to large teams.	A lot of extra code (events, states, blocs) for an app of this size.	Too heavy

We chose **Provider** with ChangeNotifier classes. The state of our app is simple: lists loaded from the API, a selected item, and loading and error flags. We use one provider per feature (authentication, students, classrooms, attendance, memorization, gifts and reports). Every provider exposes a loading flag and an error message, so every screen handle loading, errors and lost connections in the same way. Providers never contain interface code so they only use repositories and models.

5.5 Data Model

The data model is based on our EER diagram, shown below. It is normalized to Third Normal Form, so every table describes one thing and no fact is stored twice. For example, the city of each zip code is stored once in the ZipCodes table instead of being repeated for every person [8].

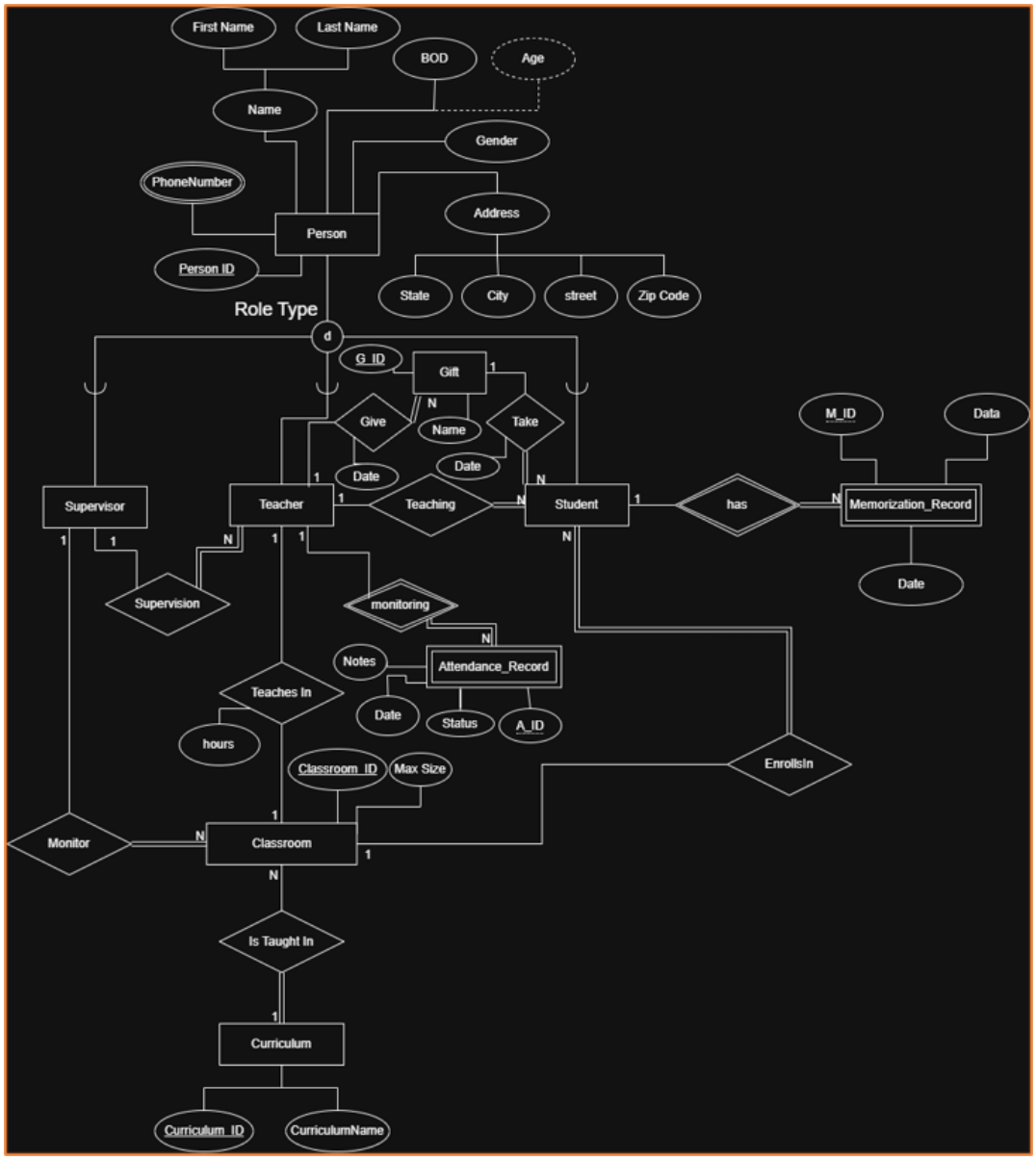


Figure 29: EER diagram of the Noor Al-Iman database

Table 18: Database tables

Table	Description
ZipCodes	City and state for each zip code.
Person	Everyone in the system: name, birth date, gender, address, phone, email and a password hash for staff.
Admin, Teacher, Supervisor, Student	Disjoint subtypes of Person that share the same PersonID. Student also has an active flag.
Curriculum	What a classroom studies, for example Juz Amma.
Classroom	Name, maximum size, teacher, supervisor and curriculum.
EnrollsIn	Which classroom each student belongs to (one classroom per student).
Attendance_Record	One record per student per day: status, note and the teacher who took it.
Memorization_Record	Weak entity of Student: Surah, verse range, evaluation, notes and date.

Table	Description
GiftType, Gift	The gift catalog, and each physical gift handed out.
GiveGift, TakenGift	Which teacher gave each gift, and which student received it, with dates.

Important rules are enforced by the API first, with a clear message for the user, and again by the database so that wrong data cannot be stored even if the API has a bug:

Table 19: Integrity rules enforced by the database

Rule	How it is enforced
A classroom cannot exceed its maximum size	Trigger check_classroom_capacity before inserting into EnrollsIn
A student with attendance cannot be deleted	Trigger prevent_student_delete before deleting from Student
A student belongs to one classroom only	Unique key on EnrollsIn.S_ID
One attendance record per student per day	Unique key on (S_ID, Date)
The verse range is valid	Check constraint: FromVerse >= 1 and ToVerse >= FromVerse
No orphan records	Foreign keys on every relationship

5.6 API Design

The API follows REST conventions. All requests and responses use JSON, every endpoint except login requires a token, and errors always return the same structure with a code and a readable message. The main groups of endpoints are listed below; the full specification is kept in the repository.

Table 20: Main API endpoints

Group	Endpoints	Who can use them
Authentication	POST /auth/login, GET /auth/me, POST /auth/change-password	All staff
Students	GET, POST /students; GET, PUT, DELETE /students/{id}; PATCH /students/{id}/active	Admin (others read their own classes)
Student history	/students/{id}/attendance, /memorization, /gifts	Scoped to the user's classrooms
Classrooms	GET, POST /classrooms; PUT /classrooms/{id}; POST, DELETE /classrooms/{id}/students	Admin (supervisors read)
Attendance	GET, PUT /classrooms/{id}/attendance?date=	Teacher of the classroom, admin
Gifts	GET, POST /gift-types; POST /students/{id}/gifts	Admin, teacher
Reports	/reports/attendance-summary, /students-without-gifts, /classes-above-average, /memorization-count	Admin, supervisor

5.7 Folder Structure

All the project's code lives in one GitHub repository with separate folders for the app, the API and the documentation:

```
noor_al_iman/
├── app/                Flutter mobile app
├── api/                PHP REST API and database scripts
├── docs/               reports, designs and diagrams
├── .github/workflows/  CI and release workflows
├── .gitignore
└── README.md
```

The Flutter app is organized by layer, with screens grouped by feature:

```
app/lib/
├── main.dart, app.dart
├── core/          config, constants, network, theme, router, utils
├── data/
│   ├── models/        one Dart class per entity
│   └── repositories/  calls to the API, return models
├── providers/     ChangeNotifier classes (state)
├── screens/       one folder per feature
└── widgets/       shared widgets
```

The PHP API follows the same idea:

```
api/
├── public/index.php   single entry point
├── routes/api.php     list of all endpoints
├── src/
│   ├── Config/        settings and database connection
│   ├── Middleware/    CORS, JSON body, authentication, roles
│   ├── Controllers/   one per feature
│   ├── Repositories/  all SQL (PDO prepared statements)
│   └── Validation/    request validation
├── database/          schema.sql and seed.sql
└── tests/             PHPUnit tests
```

5.8 GitHub Repository and Workflow

The repository is hosted at github.com/Laith98Dev/Noor-Al-Iman. The `main` branch is protected, so changes can only be merged through a pull request approved by a teammate. Work is collected on a `develop` branch, and each task has its own branch named after its issue, for example `feature/17-take-attendance`.

Commit messages follow the Conventional Commits style, such as `feat(app): add take attendance screen (#17)`, so the history shows what changed and links each commit to its issue. Because the team works together on one machine, every commit names a main author and lists the other members as co-authors, and pull requests are merged with a merge commit so each commit keeps its authors.

Two GitHub Actions workflows automate the checks. The first runs on every pull request: it analyzes and tests the Flutter app and checks and tests the PHP code, and a failing check blocks

the merge. The second runs when a version tag such as `v1.0.0` is pushed: it builds the release APK and creates a draft GitHub release with the APK attached, ready to be reviewed and published. The API itself is deployed manually: a team member connects to the VPS over SSH, pulls the latest code from GitHub and installs its dependencies.

Chapter 6: Conclusion

6.1 Summary

This report presented the first stage of the Noor Al-Iman project. We defined the problem the center faces with paper-based records and proposed a mobile application for its staff. We chose Scrum as our methodology and planned the work in sprints that follow the course schedule. The requirements were captured as user stories, functional and non-functional requirements, supported by a use case model. The UI/UX design covers every screen of the app in light and dark themes, in English and Arabic, together with the navigation flow, colours, typography and components. Finally, the architecture connects a Flutter app to a PHP REST API and a MySQL database, with Provider for state management and a GitHub repository with automated checks and releases.

6.2 Next Steps

In Stage 2 (weeks 5 to 10) we will build the database and API, then the screens feature by feature, finishing all features by the feature freeze on 25 December 2026. Stage 3 will focus on testing, performance, the release APK, the user guide and the final presentation.

References

[1] I. Sommerville, Software Engineering, 10th ed. Pearson, 2016.

[2] K. Schwaber and J. Sutherland, The Scrum Guide, 2020. https://scrumguides.org/scrum-guide.html

[3] ProjectManager, "What Is a Gantt Chart?" https://www.projectmanager.com/guides/gantt-chart

[4] Jama Software, "Requirements Gathering Techniques." https://www.jamasoftware.com/requirements-management-guide/requirements-gathering-and-management-processes/

[5] Sparx Systems, "UML 2 Tutorial: Use Case Diagram." https://sparxsystems.com/resources/tutorials/uml2/use-case-diagram.html

[6] Flutter Documentation, "List of state management approaches." https://docs.flutter.dev/data-and-backend/state-mgmt/options

[7] W3C, Web Content Accessibility Guidelines (WCAG) 2.1. https://www.w3.org/TR/WCAG21/

[8] A. Silberschatz, H. Korth and S. Sudarshan, Database System Concepts, 7th ed. McGraw-Hill, 2019.

Appendix A: Diagram Source Files

The use case diagram, navigation flow, system architecture and Scrum process diagrams were created in draw.io. Their source files are stored in the repository under `docs/diagrams/` so they can be opened and edited at app.diagrams.net.